

Databricks Lakebase

Complete Learning Guide

Architecture · Examples · Hands-On Tasks (Beginner → Advanced)

Topic	Databricks Lakebase
Version	Lakebase Autoscaling (GA Feb 2026) + LTAP (Jun 2026)
Audience	Experienced Databricks Users
Covers	Concepts, Architecture, SQL Examples, Python Integration, AI/ML Use Cases

Based on official Databricks documentation · June 2026

Table of Contents

1. What is Lakebase?
2. Architecture & Database Generation Model
3. Key Features Deep Dive
4. LTAP — Lake Transactional/Analytical Processing
5. Connecting to Lakebase
6. Working Examples
7. Unity Catalog Integration
8. Branching & Point-in-Time Restore
9. Hands-On Tasks: Beginner
10. Hands-On Tasks: Intermediate
11. Hands-On Tasks: Advanced
12. Use Cases & Patterns
13. Quick Reference Cheat Sheet

1. What is Lakebase?

Databricks Lakebase is a **fully managed, serverless PostgreSQL database** built natively into the Databricks platform. Announced at the Data + AI Summit in June 2025 and reaching General Availability in February 2026, Lakebase represents a new category of operational database — one where compute is separated from storage, data lives in open formats on cloud object storage, and the database integrates seamlessly with the Lakehouse.

Lakebase solves three long-standing pain points of traditional databases:

- **Fragile & costly operations:** Traditional monolithic databases bundle CPU and storage, requiring teams to provision for peak and run idle infrastructure.
- **Clunky developer experience:** Provisioning a dev/test copy of a production database can take hours and risk bringing production down.
- **Extreme vendor lock-in:** Proprietary storage formats trap data inside a single engine with no multi-cloud portability.

■ **Key Fact:** Lakebase handles over 12 million database launches per day across the Databricks platform (as of June 2026).

2. Architecture & Database Generation Model

Databricks frames database evolution across three generations, showing how Lakebase is the natural next step:

Generation	Examples	Storage Model	Limitations
Gen 1: Monolith	MySQL, Postgres, Oracle	CPU + Disk tightly coupled in one machine	No cloud scale, rigid provisioning, proprietary
Gen 2: Loose Coupling	Aurora, Oracle Exadata	Compute separated from storage — but proprietary format	Single-engine bottleneck, cloud lock-in, no open analytics
Gen 3: Lakebase	Databricks Lakebase	Compute separated from open object storage (Delta/Iceberg)	None of the above — open, elastic, unified

Core Architectural Principles

Compute/Storage Split	Postgres compute instances run independently of storage. Data lives in cloud object storage (S3, ADLS), allowing compute to scale to zero instantly.
------------------------------	--

Open Formats	All data is stored as Delta Lake or Apache Iceberg format — the same formats used by the Lakehouse OLAP engines.
Caching Layer	A caching layer between Postgres compute and object storage converts row data to columnar format during idle CPU time, achieving >10x compression and reducing network I/O.
Full ACID	Transactional workloads run on standard Postgres with full ACID semantics. No compromises on correctness.
Object Storage Durability	Backed by S3/ADLS — 99.999999999% (11 nines) durability, far superior to replica-based HA in traditional databases.

3. Key Features Deep Dive

- **Autoscaling**

Lakebase Autoscaling automatically adjusts Postgres compute resources based on workload demand — scaling up during spikes and down during quiet periods, including all the way to zero.

- **Scale to Zero**

When a database is idle, compute is suspended automatically. Startup on the next connection is near-instant, making Lakebase ideal for dev/test environments and bursty workloads.

- **Instant Branching**

Create isolated database branches in seconds — even for petabyte-scale databases. Branches are perfect for development, testing, schema migrations, and AI agent experimentation.

- **Point-in-Time Restore**

Restore or branch from any historical moment within your configured history window. Recover from bad migrations or data corruption without backups.

- **Read Replicas**

Create read-only replicas to scale read-heavy workloads without impacting the primary write database.

- **High Availability**

Configure automatic failover across availability zones to keep your database online during compute failures.

- **Lakehouse Sync**

Continuously replicate Lakebase Postgres tables into Unity Catalog Delta tables (SCD Type 2 history), making transactional data available for analytics and ML pipelines.

- **Synced Tables**

Sync Unity Catalog tables into Lakebase Postgres so applications can query Lakehouse data at low latency — the reverse direction of Lakehouse Sync.

- **Unity Catalog Integration**

Register Lakebase databases in Unity Catalog for unified governance, access control, and cross-source analytics.

- **Data API**

A PostgREST-compatible REST API for CRUD operations on your Lakebase tables without writing custom backend code — ideal for AI agents and serverless apps.

4. LTAP — Lake Transactional/Analytical Processing

Announced at the Data + AI Summit in June 2026, **LTAP (Lake Transactional/Analytical Processing)** is Databricks' answer to HTAP. It unifies OLTP and OLAP on a single copy of data in the lake — eliminating ETL pipelines, replicas, and data movement by design.

How LTAP Works

With LTAP, transactional data written by Lakebase Postgres lands *directly* in Delta or Iceberg format on object storage. This is the same copy that Lakehouse analytical engines (Spark, Photon, DatabricksSQL) read. There is no conversion step, no pipeline, and no lag between operational and analytical data.

Layer	Without LTAP	With LTAP
Transactional Storage	Proprietary Postgres format	Delta / Iceberg on object storage
Analytical Access	ETL pipeline needed first	Direct — zero ETL, zero lag
Data Copies	2+ copies (OLTP + OLAP)	Single copy, shared by both engines
Governance	Separate for each system	Unified in Unity Catalog
Freshness	Minutes to hours (pipeline lag)	Real-time — always in sync

■ **LTAP vs HTAP:** Traditional HTAP (Hybrid Transactional/Analytical Processing) systems unify at the engine level, still requiring data format translation. LTAP unifies at the storage layer — both engines natively read the same open-format files.

5. Connecting to Lakebase

Lakebase exposes a standard PostgreSQL wire protocol on port 5432. You can connect with any Postgres-compatible client, driver, or tool. Authentication uses Databricks OAuth tokens.

Connection via psql (CLI)

```
# Get connection details from the Lakebase UI
# Host format: .lakebase.databricks.com

export PGHOST=.lakebase.databricks.com
export PGPORT=5432
export PGDATABASE=
export PGUSER=token
export PGPASSWORD=
export PGSSLMODE=require

psql
```

Connection via Python (psycopg3)

```
import psycopg
from databricks.sdk import WorkspaceClient

# Get OAuth token via Databricks SDK
client = WorkspaceClient()
token = client.config.authenticate()["Authorization"].replace("Bearer ", "")

# Connect to Lakebase
conn = psycopg.connect(
    host=".lakebase.databricks.com",
    port=5432,
    dbname="",
    user="token",
    password=token,
    sslmode="require"
)

cursor = conn.cursor()
cursor.execute("SELECT version();")
print(cursor.fetchone())
conn.close()
```

Connection via Databricks Notebook

```
# In a Databricks notebook, use the built-in credential passthrough
import psycopg2
```

```
# Retrieve host/db from secret scope (best practice)
host = dbutils.secrets.get(scope="lakebase", key="host")
dbname = dbutils.secrets.get(scope="lakebase", key="dbname")
token = dbutils.notebook.entry_point.getDbutils().notebook().getContext().apiToken().get(
)

conn = psycopg2.connect(
    host=host,
    port=5432,
    dbname=dbname,
    user="token",
    password=token,
    sslmode="require"
)
```

Lakebase Data API (HTTP / REST)

The Data API is a PostgREST-compatible REST interface for when you do not want to manage a persistent connection pool — ideal for serverless functions and AI agents.

```
import requests

BASE_URL = "https://.databricks.com/api/2.0/lakebase/projects//data"
HEADERS = {
    "Authorization": f"Bearer {token}",
    "Content-Type": "application/json"
}

# GET rows from a table
response = requests.get(f"{BASE_URL}/orders", headers=HEADERS)
print(response.json())

# POST (insert) a new row
response = requests.post(f"{BASE_URL}/orders", headers=HEADERS,
    json={"order_id": 1001, "customer": "ACME", "amount": 549.99})
print(response.status_code)
```

6. Working Examples

Example 1: Create Tables and Basic CRUD

The following SQL runs inside the Lakebase SQL Editor or via any Postgres client.

```
-- Create an e-commerce orders table
CREATE TABLE orders (
  order_id      SERIAL PRIMARY KEY,
  customer_id   INTEGER NOT NULL,
  product_name  TEXT NOT NULL,
  quantity      INTEGER DEFAULT 1,
  unit_price    NUMERIC(10, 2),
  status        TEXT DEFAULT 'pending',
  created_at    TIMESTAMPTZ DEFAULT NOW()
);

-- Insert sample data
INSERT INTO orders (customer_id, product_name, quantity, unit_price, status)
VALUES
  (101, 'Laptop Pro 15', 1, 1299.99, 'shipped'),
  (102, 'Wireless Mouse', 2, 29.99, 'pending'),
  (103, 'USB-C Hub 7-in-1', 1, 49.99, 'delivered'),
  (101, 'Mechanical Keyboard', 1, 149.99, 'processing'),
  (104, 'Monitor 27" 4K', 2, 599.99, 'shipped');

-- Query with aggregation
SELECT
  status,
  COUNT(*) AS order_count,
  SUM(quantity * unit_price) AS total_revenue
FROM orders
GROUP BY status
ORDER BY total_revenue DESC;
```

Example 2: Sync Lakehouse Table into Lakebase (Synced Tables)

Synced Tables pull data from Unity Catalog into Lakebase for low-latency application reads.

```
-- Step 1: In Unity Catalog, create source table
CREATE TABLE main.default.user_segments AS
SELECT * FROM VALUES
  (1001, 'premium', 2500.00, 'high'),
  (1002, 'standard', 450.00, 'medium'),
  (1003, 'premium', 3200.00, 'high'),
  (1004, 'basic', 120.00, 'low')
AS t(user_id, tier, lifetime_value, engagement_level);
```

```
-- Step 2: In Lakebase SQL Editor, configure the sync
-- (Done via UI: Lakebase > Synced Tables > + New > select UC table)

-- Step 3: After sync, query in Lakebase
SELECT * FROM "default".user_segments
WHERE engagement_level = 'high'
ORDER BY lifetime_value DESC;
```

Example 3: Lakehouse Sync (Lakebase → Unity Catalog)

Lakehouse Sync replicates Lakebase Postgres changes into Delta tables with SCD Type 2 history.

```
-- Lakebase Postgres: create an operational table
CREATE TABLE inventory (
  sku          TEXT PRIMARY KEY,
  product      TEXT,
  stock_qty    INTEGER,
  updated_at   TIMESTAMPTZ DEFAULT NOW()
);

INSERT INTO inventory VALUES
  ('SKU-001', 'Widget A', 500, NOW()),
  ('SKU-002', 'Widget B', 250, NOW());

-- After enabling Lakehouse Sync in UI:
-- All changes (inserts, updates, deletes) are captured and
-- written to main.default.inventory_lakebase_sync in Unity Catalog
-- as SCD Type 2 Delta records.

-- In a Databricks notebook, analyze the change history:
-- %sql
SELECT sku, product, stock_qty, _commit_timestamp, _change_type
FROM main.default.inventory_lakebase_sync
ORDER BY _commit_timestamp DESC;
```

Example 4: Python App with Connection Pooling

```
import psycopg_pool
from databricks.sdk import WorkspaceClient

def get_token():
    client = WorkspaceClient()
    return client.config.authenticate()["Authorization"].replace("Bearer ", "")

# Create a connection pool (recommended for production apps)
pool = psycopg_pool.ConnectionPool(
    conninfo=(
        "host=.lakebase.databricks.com "
```

```

        "port=5432 dbname=myapp user=token "
        f"password={get_token()} sslmode=require"
    ),
    min_size=2,
    max_size=10,
)

def get_order(order_id: int) -> dict:
    with pool.connection() as conn:
        with conn.cursor() as cur:
            cur.execute(
                "SELECT * FROM orders WHERE order_id = %s",
                (order_id,)
            )
            row = cur.fetchone()
            cols = [d[0] for d in cur.description]
            return dict(zip(cols, row)) if row else {}

# Usage
order = get_order(1)
print(f"Order: {order}")

```

7. Unity Catalog Integration

Unity Catalog is the governance layer that ties Lakebase into the broader Databricks ecosystem. Registering your Lakebase database in Unity Catalog creates a read-only catalog that exposes Postgres schemas and tables for cross-source analytics, data lineage, and access control.

Register UC Catalog	In Lakebase UI: Settings > Unity Catalog > Register. Creates a UC catalog bound to your Postgres database.
Access Control	Grant/Revoke permissions in Unity Catalog just like any other data asset. Fine-grained access propagates to Lakebase tables.
Data Lineage	UC tracks lineage from Lakehouse tables synced into Lakebase and changes replicated back via Lakehouse Sync.
Cross-Source Queries	After registration, use Databricks SQL or Spark to JOIN Lakebase tables with Delta Lake tables in the same query.
Feature Store	Register Lakebase tables as online feature tables in Unity Catalog for real-time model serving.

```
-- After registering Lakebase in Unity Catalog,  
-- query it from a Databricks SQL warehouse:
```

```
SELECT  
  o.order_id,  
  o.product_name,  
  u.tier AS customer_tier,  
  o.unit_price * o.quantity AS total  
FROM lakebase_catalog.public.orders o  
JOIN main.default.user_segments u  
  ON o.customer_id = u.user_id  
WHERE u.tier = 'premium'  
ORDER BY total DESC;
```

8. Branching & Point-in-Time Restore

Branching is one of Lakebase's most powerful developer features. Because data lives in object storage with copy-on-write semantics, even a petabyte-scale database can be branched in seconds. Branches are fully isolated — writes on a branch do not affect production.

Branch Workflow

Production branch: The default 'main' branch — your live application data

Dev branch: Branch from main for feature development. Safe to experiment.

Test branch: Branch from a specific point-in-time for regression testing.

Merge / Reset: Drop the branch when done, or promote changes back to main via your app's migration scripts.

```
-- In the Lakebase UI: Project > Branches > + New Branch
-- Select: branch from 'main' at current HEAD (or a specific timestamp)

-- On your dev branch, run a risky schema migration:
ALTER TABLE orders ADD COLUMN discount_pct NUMERIC(5,2) DEFAULT 0;
UPDATE orders SET discount_pct = 10 WHERE customer_id IN (101, 103);

-- If something breaks, simply delete the branch and start over.
-- Production is completely unaffected.

-- Point-in-Time Restore:
-- In UI: Project > Branches > + New Branch > "From point in time"
-- Select timestamp: e.g., yesterday at 14:00 UTC
-- A new read/write branch is created at that exact state.
```

■ **Use Case:** Before running a large data cleanup or GDPR deletion job, branch from production. Validate the operation on the branch. Only then run it on the main branch.

9. Hands-On Tasks: Beginner

These tasks introduce you to core Lakebase concepts. No prior Lakebase experience needed — but you should be comfortable with SQL and the Databricks UI.

Task 1 — Create Your First Lakebase Project

Level: Beginner | Est. Time: 15 min

Objective: Create a Lakebase Autoscaling project, connect with the SQL Editor, and run your first queries.

1. In the Databricks workspace, open the app switcher and select **Lakebase**.
2. Click **+ New Project**, name it *my-first-lakebase*, and select PostgreSQL 16.
3. Wait for the project to provision (usually under 30 seconds).
4. In the project view, click **SQL Editor**.
5. Create a table and insert rows using the sample SQL below.
6. Run a SELECT query and verify the results in the output panel.

Sample Code:

```
-- Run in the Lakebase SQL Editor
CREATE TABLE IF NOT EXISTS playing_with_lakebase (
  id      SERIAL PRIMARY KEY,
  name    TEXT NOT NULL,
  value   REAL
);

-- Insert 100 random rows
INSERT INTO playing_with_lakebase (name, value)
SELECT
  LEFT(md5(i::TEXT), 10),
  random() * 1000
FROM generate_series(1, 100) AS t(i);

-- Query results
SELECT * FROM playing_with_lakebase
ORDER BY value DESC
LIMIT 10;
```

■ **Pro Tip:** The SQL Editor auto-saves your queries. Use Ctrl+Enter (Cmd+Enter on Mac) to run a selected block.

Task 2 — Explore Scale-to-Zero Behavior

Level: Beginner | Est. Time: 10 min

Objective: Observe how Lakebase automatically suspends compute when idle and resumes on demand.

1. Navigate to your Lakebase project overview and note the compute status indicator.
2. Run a query in the SQL Editor to ensure the database is active.
3. Wait 5 minutes without any activity (default idle timeout).
4. Observe the status change to *Suspended* in the project view.
5. Open the SQL Editor and run a simple `SELECT 1;`
6. Measure and record the time from query submission to first result row — this is the cold-start latency.

■ **Pro Tip:** Configure idle timeout under Project Settings > Scale-to-Zero. For production workloads, consider keeping a minimum of 1 compute unit active.

Task 3 — Connect from Python Using psycopg2

Level: Beginner | Est. Time: 20 min

Objective: Connect to Lakebase from a Python script (or notebook) and perform CRUD operations.

1. Install psycopg2: pip install psycopg2-binary
2. Retrieve your Lakebase hostname from Project Settings > Connection Details.
3. Generate a Databricks Personal Access Token (PAT) in User Settings > Developer.
4. Run the Python code below to connect and interact with Lakebase.
5. Verify the inserted rows by querying in the SQL Editor.

Sample Code:

```
import psycopg2

conn = psycopg2.connect(
    host=".lakebase.databricks.com",
    port=5432,
    dbname="",
    user="token",
    password="",
    sslmode="require"
)

cur = conn.cursor()

# Create a table
cur.execute('''
    CREATE TABLE IF NOT EXISTS products (
        id          SERIAL PRIMARY KEY,
        name        TEXT,
        price        NUMERIC(10,2),
        in_stock     BOOLEAN DEFAULT TRUE
    )
''')

# Insert rows
products = [
    ("Widget Alpha", 19.99, True),
    ("Widget Beta", 34.99, True),
    ("Widget Gamma", 9.99, False),
]

cur.executemany(
    "INSERT INTO products (name, price, in_stock) VALUES (%s, %s, %s)",
    products
)
```

```
conn.commit()

# Read back
cur.execute("SELECT * FROM products WHERE in_stock = TRUE")
for row in cur.fetchall():
    print(row)

cur.close()
conn.close()
```

■ **Pro Tip:** Never hardcode credentials. Use environment variables or Databricks Secrets:
`dbutils.secrets.get(scope='my-scope', key='lakebase-pat')`

10. Hands-On Tasks: Intermediate

These tasks assume you can connect to Lakebase and run basic SQL. They focus on the features that differentiate Lakebase from a plain Postgres database.

Task 4 — Database Branching for Safe Schema Migration

Level: Intermediate | Est. Time: 30 min

Objective: Use branching to test a destructive schema migration before applying it to production.

1. In your Lakebase project, ensure you have a populated table (use Task 1 or 3 data).
2. In the Lakebase UI, go to **Branches** and click **+ New Branch**. Name it *schema-migration-test*.
3. Connect to the new branch (the SQL Editor lets you select the active branch).
4. Run the migration SQL below on the branch.
5. Validate that the migration looks correct by querying the branch.
6. If satisfied, run the same migration on the main branch. If not, simply delete the branch.

Sample Code:

```
-- Run on the branch: schema-migration-test

-- Simulate a risky migration: rename a column + add NOT NULL constraint
ALTER TABLE products RENAME COLUMN name TO product_name;
ALTER TABLE products ADD COLUMN category TEXT;
UPDATE products SET category = 'general' WHERE category IS NULL;
ALTER TABLE products ALTER COLUMN category SET NOT NULL;

-- Validate
SELECT product_name, price, category, in_stock
FROM products LIMIT 5;

-- Optionally: drop the branch via UI if migration needs revision
-- Or: re-run the same SQL on 'main' branch if it looks good
```

■ **Pro Tip:** Treat Lakebase branches like Git branches — one branch per migration ticket. This gives you a full audit trail of what was tested.

Task 5 — Sync a Unity Catalog Table into Lakebase

Level: Intermediate | Est. Time: 25 min

Objective: Use Synced Tables to pull a Unity Catalog Delta table into Lakebase for low-latency app reads.

1. In a Databricks notebook, create a Unity Catalog source table with the SQL below.
2. In the Lakebase UI, go to **Synced Tables** and click **+ New**.
3. Select your Unity Catalog table: *main.default.product_catalog*.
4. Configure sync options: Full refresh or incremental. Click **Start Sync**.
5. Once sync completes, switch to the Lakebase SQL Editor and query the synced table.
6. Update a row in the Unity Catalog source and trigger a re-sync. Verify the change appears in Lakebase.

Sample Code:

```
-- Step 1: Run in a Databricks Notebook (Unity Catalog)
CREATE OR REPLACE TABLE main.default.product_catalog AS
SELECT * FROM VALUES
  ('P001', 'Laptop Pro', 'Electronics', 1299.99, 4.5),
  ('P002', 'Office Chair', 'Furniture', 349.99, 4.2),
  ('P003', 'Coffee Maker', 'Appliances', 89.99, 4.7),
  ('P004', 'Desk Lamp', 'Furniture', 39.99, 4.0),
  ('P005', 'Wireless Headphones', 'Electronics', 199.99, 4.8)
AS t(sku, product_name, category, price, rating);

-- Step 2: After sync, run in Lakebase SQL Editor
SELECT category, COUNT(*) AS products, AVG(rating) AS avg_rating
FROM "default".product_catalog
GROUP BY category
ORDER BY avg_rating DESC;
```

■ **Pro Tip:** Synced Tables are read-only in Lakebase. For write-back scenarios (e.g., enriching data and sending it back to UC), use Lakehouse Sync.

Task 6 — Enable Lakehouse Sync (OLTP → Delta)

Level: Intermediate | Est. Time: 35 min

Objective: Continuously replicate Lakebase Postgres changes to a Unity Catalog Delta table for downstream analytics.

1. Create an operational table in Lakebase with the SQL below.
2. Insert initial rows and commit.
3. In the Lakebase UI, go to **Lakehouse Sync** and enable sync for the *transactions* table.
4. Select the target UC catalog, schema, and table name.
5. In a Databricks notebook, read the synced Delta table using the SQL below.
6. Insert more rows in Lakebase, wait ~30 seconds, and re-query the Delta table to see the changes.

Sample Code:

```
-- Step 1: In Lakebase SQL Editor
CREATE TABLE transactions (
  txn_id      SERIAL PRIMARY KEY,
  account_id  INTEGER NOT NULL,
  amount      NUMERIC(12, 2),
  txn_type    TEXT CHECK (txn_type IN ('debit', 'credit')),
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

INSERT INTO transactions (account_id, amount, txn_type) VALUES
  (1001, 500.00, 'credit'),
  (1002, 200.00, 'debit'),
  (1001, 75.50, 'debit');

-- Step 2: In Databricks Notebook after enabling sync
-- %sql
SELECT
  account_id,
  SUM(CASE WHEN txn_type = 'credit' THEN amount ELSE -amount END) AS balance,
  COUNT(*) AS num_transactions
FROM main.default.transactions_sync
GROUP BY account_id;
```

■ **Pro Tip:** Lakehouse Sync uses SCD Type 2 — every row change is appended with a timestamp and change type (`_change_type`). Use this for full audit trails.

11. Hands-On Tasks: Advanced

These tasks involve multi-component architectures, AI/ML integration, and production-grade patterns. Familiarity with Databricks Apps, MLflow, and Unity Catalog is helpful.

Task 7 — Build a Transactional App Backend with Databricks Apps

Level: Advanced | Est. Time: 60 min

Objective: Use Databricks Apps + Lakebase to build a production-ready CRUD application with OAuth-authenticated API access.

1. Create a Databricks App (Apps > + New App) from the blank Python/Flask template.
2. In the app's `app.py`, implement the connection pool and REST endpoints below.
3. Bind the app to your Lakebase project via App Settings > Databases.
4. Deploy the app and test the `/orders` GET and POST endpoints.
5. Add error handling and input validation for production hardening.
6. Monitor query latency in the Lakebase Query Insights panel.

Sample Code:

```
# app.py – Databricks App backend
import os
import psycopg_pool
from flask import Flask, request, jsonify

app = Flask(__name__)

# Databricks Apps injects DB credentials as env vars when bound to Lakebase
pool = psycopg_pool.ConnectionPool(
    conninfo=(
        f"host={os.environ['LAKEBASE_HOST']} "
        f"port=5432 dbname={os.environ['LAKEBASE_DB']} "
        f"user=token password={os.environ['DATABRICKS_TOKEN']} "
        "sslmode=require"
    ),
    min_size=2, max_size=20
)

@app.route("/orders", methods=["GET"])
def list_orders():
    with pool.connection() as conn:
        cur = conn.cursor()
        cur.execute("SELECT * FROM orders ORDER BY created_at DESC LIMIT 50")
        cols = [d[0] for d in cur.description]
        rows = [dict(zip(cols, r)) for r in cur.fetchall()]
    return jsonify(rows)

@app.route("/orders", methods=["POST"])
def create_order():
    data = request.json
    with pool.connection() as conn:
```

```
cur = conn.cursor()
cur.execute(
    "INSERT INTO orders (customer_id, product_name, quantity, unit_price) "
    "VALUES (%s, %s, %s, %s) RETURNING order_id",
    (data["customer_id"], data["product_name"],
     data["quantity"], data["unit_price"])
)
order_id = cur.fetchone()[0]
conn.commit()
return jsonify({"order_id": order_id}), 201

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)
```

■ **Pro Tip:** Databricks Apps automatically handle OAuth token refresh when bound to Lakebase — you do not need to implement token rotation manually.

Task 8 — Lakebase as an Online Feature Store for ML Models

Level: Advanced | Est. Time: 75 min

Objective: Use Lakebase as the online serving layer for ML features computed offline in the Lakehouse, then serve them to a real-time model.

1. Compute feature aggregations in a Databricks notebook using Spark (code below).
2. Write the features to a Lakebase table using psycopg2 from the notebook.
3. Register the Lakebase table as an Online Feature Table in Unity Catalog.
4. In a Model Serving endpoint, fetch features from Lakebase at inference time using the Data API.
5. Measure p99 feature fetch latency — it should be under 5ms for primary key lookups.
6. Set up a scheduled Databricks Job to recompute and refresh features daily.

Sample Code:

```
# Notebook cell 1: Compute features offline with Spark
features_df = spark.sql(
    "SELECT customer_id,"
    " COUNT(*) AS order_count_30d,"
    " SUM(unit_price * quantity) AS spend_30d,"
    " AVG(unit_price * quantity) AS avg_order_value,"
    " MAX(created_at) AS last_order_ts"
    " FROM main.default.orders_history"
    " WHERE created_at >= current_date() - INTERVAL 30 DAYS"
    " GROUP BY customer_id"
)

# Notebook cell 2: Write to Lakebase
import psycopg2, psycopg2.extras

rows = features_df.collect()
conn = psycopg2.connect(host=HOST, port=5432, dbname=DB,
                        user="token", password=TOKEN, sslmode="require")
cur = conn.cursor()
cur.execute(
    "CREATE TABLE IF NOT EXISTS customer_features ("
    "  customer_id      INTEGER PRIMARY KEY,"
    "  order_count_30d  INTEGER,"
    "  spend_30d        NUMERIC(12,2),"
    "  avg_order_value  NUMERIC(12,2),"
    "  last_order_ts    TIMESTAMPTZ,"
    "  refreshed_at     TIMESTAMPTZ DEFAULT NOW()"
    ")"
)
psycopg2.extras.execute_values(cur,
    "INSERT INTO customer_features VALUES %s ON CONFLICT (customer_id) DO UPDATE "
```

```
"SET order_count_30d=EXCLUDED.order_count_30d, spend_30d=EXCLUDED.spend_30d, "  
"avg_order_value=EXCLUDED.avg_order_value, last_order_ts=EXCLUDED.last_order_ts, "  
"refreshed_at=EXCLUDED.refreshed_at",  
[(r.customer_id, r.order_count_30d, r.spend_30d,  
  r.avg_order_value, r.last_order_ts) for r in rows]  
)  
conn.commit()  
print(f"Upserted {len(rows)} feature rows to Lakebase")
```

■ **Pro Tip:** Use ON CONFLICT DO UPDATE (upsert) to keep feature tables fresh. For sub-millisecond lookups, create indexes: CREATE INDEX ON customer_features (customer_id);

Task 9 — Implement an AI Agent State Store with LTAP Analytics

Level: Advanced | Est. Time: 90 min

Objective: Build an AI agent that persists conversation state and decisions in Lakebase, then use LTAP to run real-time analytics on agent activity in Databricks SQL — all from a single copy of data with no ETL.

1. Create the agent state tables in Lakebase using the SQL below.
2. Write a Python class that wraps Lakebase for agent state persistence.
3. Enable LTAP (Lakehouse Sync) on both tables so they land in Unity Catalog as Delta.
4. In Databricks SQL, run analytical queries on agent activity directly on the Delta tables.
5. Create a Databricks Dashboard to monitor agent decision patterns in real time.
6. Add a Lakebase read replica for the analytics query path to isolate it from agent writes.

Sample Code:

```
-- Step 1: Lakebase schema for AI agent state
CREATE TABLE agent_sessions (
  session_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_name  TEXT NOT NULL,
  user_id     TEXT,
  started_at  TIMESTAMPTZ DEFAULT NOW(),
  last_active TIMESTAMPTZ DEFAULT NOW(),
  status      TEXT DEFAULT 'active'
);

CREATE TABLE agent_events (
  event_id     BIGSERIAL PRIMARY KEY,
  session_id   UUID REFERENCES agent_sessions(session_id),
  event_type   TEXT NOT NULL, -- 'thought', 'tool_call', 'decision', 'error'
  content      JSONB,
  tokens_used  INTEGER DEFAULT 0,
  latency_ms   INTEGER,
  created_at   TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX ON agent_events (session_id, created_at DESC);
CREATE INDEX ON agent_events USING gin (content); -- JSONB full-text search

-- Step 2: Python agent state manager
import pycpg2, json
from uuid import uuid4

class AgentStateStore:
  def __init__(self, conn):
    self.conn = conn
```

```

def new_session(self, agent_name, user_id=None):
    cur = self.conn.cursor()
    cur.execute(
        "INSERT INTO agent_sessions (agent_name, user_id) VALUES (%s, %s) RETURNING session_id",
        (agent_name, user_id)
    )
    session_id = cur.fetchone()[0]
    self.conn.commit()
    return str(session_id)

def log_event(self, session_id, event_type, content, tokens=0, latency_ms=0):
    cur = self.conn.cursor()
    cur.execute(
        "INSERT INTO agent_events (session_id, event_type, content, tokens_used, latency_ms) "
        "VALUES (%s, %s, %s, %s, %s)",
        (session_id, event_type, json.dumps(content), tokens, latency_ms)
    )
    cur.execute(
        "UPDATE agent_sessions SET last_active = NOW() WHERE session_id = %s",
        (session_id,)
    )
    self.conn.commit()

-- Step 3: Analytics in Databricks SQL (via LTAP / Lakehouse Sync)
-- %sql
SELECT
    agent_name,
    event_type,
    COUNT(*) AS event_count,
    AVG(latency_ms) AS avg_latency_ms,
    SUM(tokens_used) AS total_tokens
FROM main.default.agent_events_sync ae
JOIN main.default.agent_sessions_sync s ON ae.session_id = s.session_id
WHERE ae.created_at >= current_timestamp() - INTERVAL 1 HOUR
GROUP BY agent_name, event_type
ORDER BY event_count DESC;

```

■ **Pro Tip:** Use JSONB GIN indexes in Lakebase for fast content searches across agent event payloads. In Unity Catalog, the same data is available as a VARIANT column in Delta format.

12. Use Cases & Patterns

App Backend + Lakehouse Analytics

Build a transactional application backend in Lakebase while the same data is available for complex analytics in Databricks SQL — eliminating the need for a separate data warehouse sync.

Online Feature Store

Compute ML features in batch using Spark. Write them to Lakebase for sub-millisecond primary-key lookups at model serving time. Refresh features on a schedule.

AI Agent State Store

AI agents need fast, ACID-compliant storage for conversation history, tool call logs, and decision states. Lakebase provides this with full Postgres semantics.

Dev/Test Database Branching

Replace slow, expensive dev database provisioning with instant Lakebase branches. Each developer or CI job gets an isolated copy of production data in seconds.

Real-Time Inventory Management

Write inventory updates from POS or warehouse systems into Lakebase, with Lakehouse Sync feeding the Delta Lake for demand forecasting and reorder models.

GDPR / Data Cleanup Workflows

Branch from production, run deletion queries on the branch, verify results, then replay the same commands on main with confidence.

13. Quick Reference Cheat Sheet

Task	How	Notes
Create project	Lakebase UI > + New Project	Choose Autoscaling tier
Connect via psql	psql -h -U token -d	Set PGPASSWORD to PAT/OAuth
Connect Python	psycopg2/psycopg3 with sslmode=require	Use connection pool for apps
Create branch	UI: Branches > + New Branch	Instant, even at petabyte scale
Point-in-time restore	UI: Branches > From point in time	Creates a new branch at that state
Sync UC → Lakebase	UI: Synced Tables > + New	Read-only in Lakebase
Sync Lakebase → UC	UI: Lakehouse Sync > Enable	SCD Type 2 Delta output
Register in UC	Settings > Unity Catalog > Register	Enables cross-source queries
REST / Data API	POST /api/2.0/lakebase/.../data/	PostgreSQL-compatible
Autoscaling	Enabled by default on new projects	Scales to 0 on idle
High Availability	Settings > High Availability > Enable	Auto-failover across AZs
Read replicas	UI: Read Replicas > + New Replica	For read-heavy analytics queries
LTAP	Requires Lakebase Autoscaling + UC	Single copy, zero ETL, zero lag

Key Databricks Documentation Links

- Lakebase Postgres Overview docs.databricks.com/aws/en/oltp/projects/
- Get Started with Lakebase docs.databricks.com/aws/en/oltp/projects/get-started
- Lakehouse Sync docs.databricks.com/aws/en/oltp/projects/lakehouse-sync
- Synced Tables docs.databricks.com/aws/en/oltp/projects/sync-tables
- Lakebase Data API docs.databricks.com/aws/en/oltp/projects/data-api
- Lakebase Branches docs.databricks.com/aws/en/oltp/projects/branches
- LTAP Announcement databricks.com/company/newsroom/press-releases/databricks-launches-ltap-first-lake-transactional-analytical

- Lakebase Blog Post

databricks.com/blog/what-is-a-lakebase

Generated June 2026 · Sources: Databricks official documentation and blog posts